JGI JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Spring Boot AOP, Runners, and Log

Table of Contents

UNIT 14: Spring Boot AOP, Runners, and Log

Objectives:

In this Unit you will learn about –

- Key concepts in Aspect-Oriented Programming (AOP)
- Spring Boot Runners
- Logging in Spring Boot

Learning Outcome:

Gain an introduction to aspect oriented programming.

14. Spring Boot AOP, Runners, and Log

1. Aspect-Oriented Programming (AOP)

Aspect-Oriented Programming (AOP) is a programming paradigm that aims to modularize cross-cutting concerns in a software application. Cross-cutting concerns are aspects of the application that affect multiple parts of the codebase, such as logging, security, and transactions. AOP provides a way to separate these concerns from the core business logic, promoting better code organization, reusability, and maintainability.

AOP achieves this by allowing developers to encapsulate these concerns, called "aspects," separately from the main application logic. These aspects are then "woven" into the application's code at various points, called join points, to achieve the desired behavior. AOP helps reduce code duplication, promotes separation of concerns, and makes the codebase more manageable.

Key Concepts in AOP

- Aspect: An aspect is a module that encapsulates a cross-cutting concern, such as logging or security policies. It defines the behavior that should be applied at specific join points in the code.
- Advice: Advice is the actual action or behavior that is executed at a particular join point. It represents the logic associated with an aspect, such as code executed before or after a method call.
- Join Point: A join point is a specific point in the execution of a program, such as method invocation or field access, where an aspect can be applied. Join points are defined by specific criteria, called pointcuts.
- Pointcut: A pointcut defines a set of join points where an aspect's advice should be applied. It specifies the conditions under which an aspect's behavior is triggered.
- Weaving: Weaving is the process of integrating aspects into the main

application codebase at the specified join points. It can be done at different times, such as at compile time, load time, or runtime.

Types of Advice in AOP

a. Before Advice

Before advice is executed before the join point, typically before a method invocation. It is used to perform actions such as logging, security checks, or parameter validation before the actual method execution.

In Spring Boot, before advice can be implemented using annotations such as @Before from the org.aspectj.lang.annotation package. The advice method is annotated with @Before and specifies the pointcut expression to define where the advice should be applied.

b. After Advice

After advice is executed after the join point, regardless of whether the method execution completed successfully or threw an exception. It is often used for actions like cleanup, resource release, or auditing.

Spring Boot provides annotations like @After, @AfterReturning, and @AfterThrowing for implementing after advice. These annotations are used to define methods that should be executed after a specific pointcut.

c. Around Advice

Around advice wraps around the join point, allowing you to control the method's execution. It can modify method arguments, skip method execution, or perform custom behavior before and after the method call.

To implement around advice in Spring Boot, the @Around annotation is used. The advice method receives a ProceedingJoinPoint parameter, which allows you to invoke the wrapped method and control its execution.

d. Returning Advice

Returning advice is executed after a successful method execution. It allows you to access the method's return value and perform actions based on it, such as logging the returned result.

Spring Boot provides the @AfterReturning annotation for implementing returning advice. The annotated method is executed after a successful method execution, and you can access the return value using the method's parameter.

e. Throwing Advice

Throwing advice is executed if a method throws an exception. It provides a way to handle exceptions, log errors, or perform cleanup operations in response to exceptions.

Spring Boot's @AfterThrowing annotation is used for implementing throwing advice. The annotated method is executed when a method throws an exception, and you can access the thrown exception using the method's parameter.

Creating Aspects

Creating Custom Aspects Using Annotations and XML Configuration

1. Annotation-Based Configuration: Spring Boot supports creating aspects using annotations like @Aspect and advice annotations such as @Before, @After, @Around, etc. Annotate a class with @Aspect to indicate it as an aspect, and define advice methods within it.
2. XML-Based Configuration: Aspects can also be defined using XML configuration. Configure aspect beans, pointcuts, and advice in the Spring Boot application context XML file.

Aspects often require a combination of different types of advice to achieve the desired behavior. For example, a logging aspect might use before and after advice to log method entry and exit, and throwing advice to handle exceptions. By combining

different types of advice, you can address multiple concerns in a single aspect. This promotes cleaner code organization and better separation of concerns.

2. Spring Boot Runners

Application Runner

The ApplicationRunner interface in Spring Boot is used to perform specific tasks when a Spring Boot application starts up. It provides a way to execute custom logic after the Spring application context has been fully initialized.

To implement the ApplicationRunner interface, follow these steps:

1. Create a class that implements the ApplicationRunner interface.
2. Implement the run method from the interface. This method contains the code that will be executed when the application starts.

Example of implementing the ApplicationRunner:

```
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

@Component
public class MyApplicationRunner implements ApplicationRunner {

    @Override
    public void run(ApplicationArguments args) throws Exception {
        // Custom logic to execute on application startup
        System.out.println("ApplicationRunner executed on startup");
```

```
    }
}
```

Use cases for the ApplicationRunner:

- Database initialization: You can use it to perform database migrations or data seeding when the application starts.
- External system connections: Establish connections to external services or APIs upon application startup.
- Cache warming: Preload data into caches for improved performance.

Command-Line Runner

The CommandLineRunner interface in Spring Boot is similar to ApplicationRunner but is specifically designed to execute code via the command line. It allows you to pass command-line arguments to your Spring Boot application and perform certain tasks during application startup.

To implement the CommandLineRunner interface, follow these steps:

1. Create a class that implements the CommandLineRunner interface.
2. Implement the run method, which takes an array of strings representing the command-line arguments.

Example of implementing the CommandLineRunner:

```
import org.springframework.boot.CommandLineRunner;

import org.springframework.stereotype.Component;


@Component

public class MyCommandLineRunner implements CommandLineRunner {
```

```
    @Override
    public void run(String... args) throws Exception {
        // Custom logic to execute based on command-line arguments
        System.out.println("CommandLineRunner executed with arguments: " +
 String.join(", ", args));
    }
}
```

Use cases for the CommandLineRunner include:

- Configuration setup: Perform initial configuration based on command-line arguments.
- Batch processing: Execute batch jobs or data processing tasks via the command line.
- Diagnostic tasks: Run diagnostic checks or system health checks on application startup.

3. Logging in Spring Boot

Logging is a critical practice in software development for several reasons:

- Debugging and Troubleshooting: Logs provide insights into application behavior, helping developers identify and fix issues during development and in production environments.
- Monitoring and Maintenance: Logs allow system administrators to monitor application health, detect anomalies, and perform maintenance tasks.
- Auditing and Compliance: Logs capture important events and transactions, aiding in auditing, compliance, and legal requirements.
- Performance Analysis: Logging can help analyze application performance,

identify bottlenecks, and optimize resource usage.

There are various logging frameworks available for Java applications, including:

1. Logback: A successor to Log4j, Logback is widely used and is the default logging framework in Spring Boot.
2. Log4j: An older and popular logging framework, known for its flexibility and configurability.
3. java.util.logging (JUL): A built-in logging framework provided by the Java platform.
4. SLF4J (Simple Logging Facade for Java): A logging facade or wrapper that provides a common API for various logging frameworks, allowing you to switch between them easily.

Log Levels

Logging frameworks define several log levels to categorize the importance of log messages:

- TRACE: Detailed diagnostic information, typically used for debugging purposes.
- DEBUG: Fine-grained information useful for debugging during development.
- INFO: Informational messages about application state and events.
- WARN: Indication of potential issues that might need attention.
- ERROR: Errors or exceptions that indicate a problem that needs fixing.
- FATAL: Severe errors that might lead to application failure.

Choosing the appropriate log level depends on the context:

1. Development: Use DEBUG and TRACE levels for detailed debugging.

2. Production: Use INFO for general information, WARN for potential issues, and ERROR for critical errors.

Configuring Logging in Spring Boot

a. Console Log Output

- Configuring Console Output: Spring Boot provides default console logging. You can configure it by adjusting properties in the application.properties or application.yml file.
- Formatting Log Messages: Customize log message format using placeholders, timestamps, log levels, and more.

b. File Log Output

- Configuring Log File Output: Spring Boot allows you to redirect logs to a file. Configure log file properties and locations in the application configuration.
- Managing Log File Rotation and Size: Configure log rotation strategies, maximum file sizes, and the number of backup log files to manage log files effectively.